

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

WEEK 12

SEARCHING



Disusun oleh:

MANGGALA PATRA RADITYA

109082500179

S1IF-13-02

Asisten Praktikum

Muhammad Agha Zulfadhli Renisa

Assyifa Putri

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

WEEK 12

1. Soal 1

Source code

```
package main

import "fmt"

func main() {
    var suara [21]int
    var x int
    total := 0
    valid := 0

    for {
        fmt.Scan(&x)

        if x == 0 {
            break
        }

        total++

        if x >= 1 && x <= 20 {
            suara[x]++
            valid++
        }
    }

    fmt.Println("Suara masuk:", total)
    fmt.Println("Suara sah:", valid)

    for i := 1; i <= 20; i++ {
        if suara[i] > 0 {
            fmt.Printf("%d: %d\n", i, suara[i])
        }
    }
}
```

Screenshoot program

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var suara [21]int
7     var x int
8     total := 0
9     valid := 0
10
11     for {
12         fmt.Scan(&x)
13
14         if x == 0 {
```

```
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING\Soal 1\soal1.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING>
```

Deskripsi program

Program pada Soal no 1 bertujuan untuk menghitung hasil pemungutan suara pemilihan ketua RT dengan membaca data nomor calon yang dipilih hingga ditemukan angka **0** sebagai penanda akhir input. Setiap data yang dibaca dihitung sebagai **suara masuk**, kemudian divalidasi apakah berada pada rentang **1–20** sebagai nomor calon yang sah. Jika valid, program menambahkan jumlah **suara sah** serta menyimpan perolehan suara masing-masing calon menggunakan sebuah array, di mana indeks array merepresentasikan nomor calon dan nilainya menunjukkan jumlah suara yang diperoleh. Setelah seluruh data selesai dibaca, program menampilkan jumlah suara masuk, jumlah suara sah, serta daftar calon yang memperoleh suara beserta total suaranya. Pendekatan ini memanfaatkan array sebagai pencatat frekuensi sehingga proses penghitungan suara menjadi sederhana, efisien, dan hanya memerlukan satu kali pembacaan data (**O(n)**).

2. Soal 2

Source code

```
package main

import "fmt"

func main() {
    var suara [21]int
    var x int
    total := 0
    valid := 0

    for {
        fmt.Scan(&x)

        if x == 0 {
            break
        }

        total++

        if x >= 1 && x <= 20 {
            suara[x]++
            valid++
        }
    }

    ketua := 1
    wakil := 2

    for i := 2; i <= 20; i++ {
        if suara[i] > suara[ketua] || (suara[i] == suara[ketua] &&
i < ketua) {
```

```
        ketua = i
    }
}

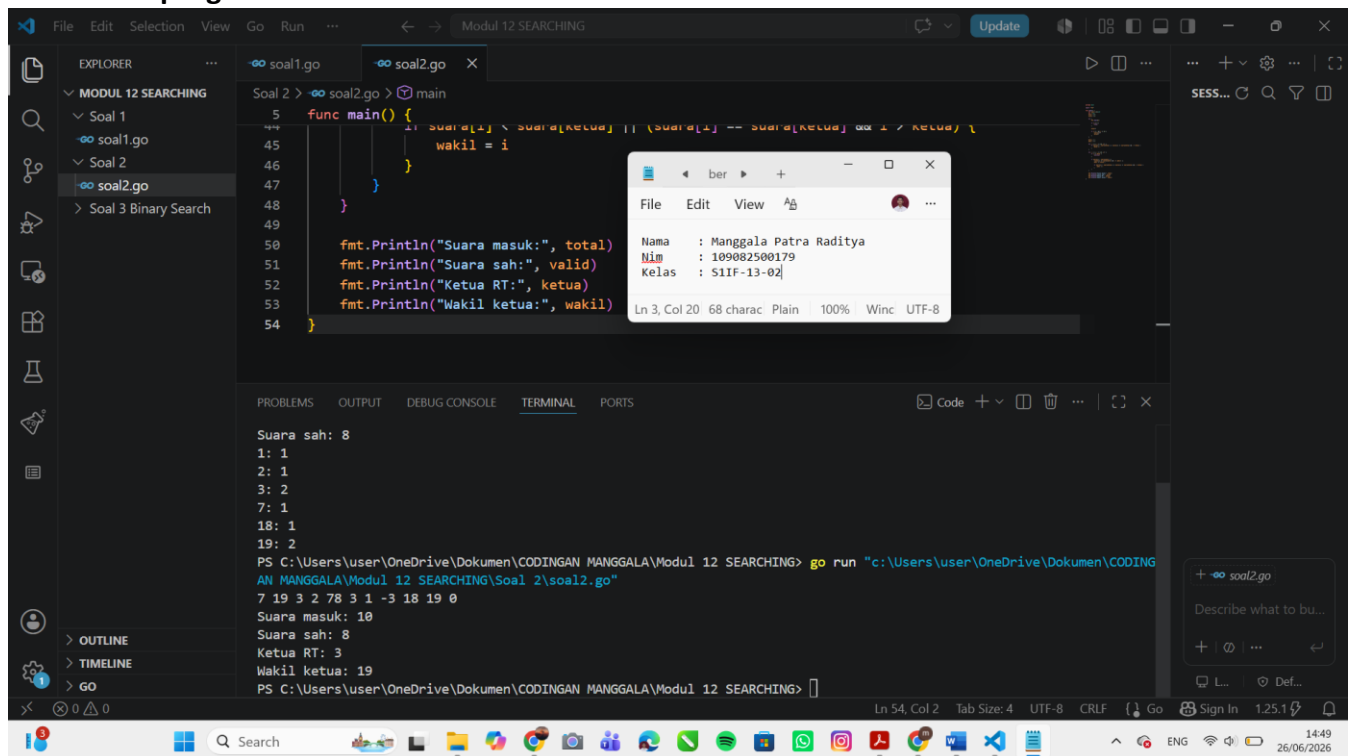
for i := 1; i <= 20; i++ {
    if i == ketua {
        continue
    }

    if suara[i] > suara[wakil] ||
        (suara[i] == suara[wakil] && i < wakil) ||
        wakil == ketua {

        if suara[i] < suara[ketua] || (suara[i] ==
suara[ketua] && i > ketua) {
            wakil = i
        }
    }
}

fmt.Println("Suara masuk:", total)
fmt.Println("Suara sah:", valid)
fmt.Println("Ketua RT:", ketua)
fmt.Println("Wakil ketua:", wakil)
}
```

Screenshoot program



```
func main() {
    5
    45
    46
    47
    48
    49
    50
    51
    52
    53
    54

    fmt.Println("Suara masuk:", total)
    fmt.Println("Suara sah:", valid)
    fmt.Println("Ketua RT:", ketua)
    fmt.Println("Wakil ketua:", wakil)
}

// Output:
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING\Soal 2\soal2.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING>
```

Deskripsi program

Pada Soal no 2, program bertujuan untuk menentukan **ketua RT dan wakil ketua RT** berdasarkan hasil perolehan suara dari 20 calon peserta. Program membaca data nomor calon yang dipilih warga satu per satu hingga menemukan angka **0** sebagai penanda akhir masukan. Setiap data yang dibaca dihitung sebagai **suara masuk**, kemudian dilakukan proses validasi untuk memastikan bahwa nomor calon berada pada rentang **1 sampai 20**. Apabila nomor tersebut valid, maka jumlah **suara sah** akan ditambah dan perolehan suara calon yang bersangkutan disimpan dalam sebuah array, di mana setiap indeks array mewakili nomor calon dan setiap elemennya menyimpan jumlah suara yang diperoleh. Setelah seluruh data selesai diproses, program melakukan pencarian calon dengan jumlah suara terbanyak untuk ditetapkan sebagai **Ketua RT**. Selanjutnya program kembali melakukan pencarian untuk menentukan calon dengan jumlah suara terbanyak kedua sebagai **Wakil Ketua RT**, dengan mengabaikan calon yang telah terpilih sebagai ketua. Jika terdapat dua atau lebih calon yang memperoleh jumlah suara yang sama, maka penentuan pemenang dilakukan berdasarkan aturan bahwa calon dengan **nomor peserta yang lebih kecil** memiliki prioritas lebih tinggi. Aturan yang sama juga diterapkan dalam pemilihan wakil ketua apabila terjadi jumlah suara yang sama pada posisi kedua. Terakhir, program menampilkan jumlah suara masuk, jumlah suara sah, nomor calon yang terpilih sebagai Ketua RT, serta nomor calon yang terpilih sebagai Wakil Ketua RT. Algoritma yang digunakan memanfaatkan array untuk menyimpan frekuensi suara setiap calon dan melakukan pencarian secara berurutan (**sequential search**) terhadap seluruh data perolehan suara untuk menemukan nilai maksimum dan maksimum kedua, sehingga seluruh proses dapat diselesaikan secara efisien dengan kompleksitas waktu **O(n)** terhadap jumlah data suara yang dibaca.

3. Soal 3

Source code

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kiri := 0
    kanan := n - 1

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if data[tengah] == k {
            return tengah
        } else if data[tengah] < k {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }

    return -1
}
```

```
}

func main() {
    var n, k int

    fmt.Scan(&n, &k)

    isiArray(n)

    hasil := posisi(n, k)

    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}
```


Screenshoot program

The screenshot shows a Go IDE with a project named 'Modul 12 SEARCHING'. The Explorer pane on the left shows the project structure with files 'soal1.go', 'soal2.go', and 'soal3_binary_search.go'. The main editor displays the code for 'soal3_binary_search.go', which includes a `main` function and a `posisi` function. A small window titled 'ber' is open, displaying the output of the program: 'Nama : Manggala Patra Raditya', 'Nilai : 109082500179', and 'Kelas : SIIF-13-02'. The terminal pane at the bottom shows the command `go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING\soal3_binary_search.go"` and the output of the program, which includes the input values and the result 'TIDAK ADA'.

```
func main() {  
    34  
    35  
    36     fmt.Scan(&n, &k)  
    37  
    38     isiArray(n)  
    39  
    40     hasil := posisi(n, k)  
    41  
    42     if hasil == -1 {  
    43         fmt.Println("TIDAK ADA")  
    44     } else {  
    45         fmt.Println(hasil)  
    46     }  
    47 }  
    48 }
```

```
AN MANGGALA\Modul 12 SEARCHING\Soal 3 Binary Search\soal3_binary_search.go"  
12 534  
1 3 8 16 32 123 323 323 534 543 823 999  
8  
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING> go run "c:\Users\user\OneDrive\Dokumen\CODING  
AN MANGGALA\Modul 12 SEARCHING\Soal 3 Binary Search\soal3_binary_search.go"  
12 535  
1 3 8 16 32 123 323 323 534 543 823 999  
TIDAK ADA  
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 12 SEARCHING>
```

Deskripsi program

Pada Soal binary search no 3, program yang dibuat bertujuan untuk mencari keberadaan sebuah bilangan bulat **k** di dalam sekumpulan data integer positif yang **sudah terurut secara menaik (ascending)** dengan memanfaatkan algoritma **Binary Search**. Program menerima masukan dalam dua baris, di mana baris pertama berisi dua buah bilangan, yaitu **n** yang menyatakan banyaknya data (dengan batas maksimum 1.000.000 elemen) dan **k** sebagai nilai yang akan dicari, sedangkan baris kedua berisi **n** buah bilangan bulat positif yang telah tersusun dari nilai terkecil hingga terbesar. Setelah membaca nilai **n** dan **k**, program memanggil prosedur `isiArray(n)` untuk mengisi array global data dengan seluruh elemen yang diberikan. Selanjutnya, fungsi `posisi(n, k)` melakukan proses pencarian menggunakan metode Binary Search dengan menetapkan dua buah penunjuk, yaitu **kiri** yang diawali pada indeks pertama (0) dan **kanan** yang diawali pada indeks terakhir ($n-1$). Selama nilai **kiri** masih lebih kecil atau sama dengan **kanan**, program menghitung indeks tengah ($\text{tengah} = (\text{kiri} + \text{kanan}) / 2$) sebagai titik pembanding. Apabila nilai pada indeks tengah sama dengan nilai **k**, maka fungsi langsung mengembalikan indeks tersebut sebagai posisi ditemukannya data. Jika nilai tengah lebih kecil daripada **k**, berarti data yang dicari hanya mungkin berada di bagian kanan array karena seluruh data telah terurut menaik, sehingga batas kiri digeser menjadi $\text{tengah} + 1$. Sebaliknya, apabila nilai tengah lebih besar daripada **k**, maka pencarian dipersempit ke bagian kiri dengan menggeser batas kanan menjadi $\text{tengah} - 1$. Proses ini berlangsung berulang hingga data ditemukan atau hingga batas kiri melewati batas kanan, yang menandakan bahwa nilai **k** tidak terdapat di dalam array. Jika data ditemukan, program menampilkan **indeks** data tersebut dengan perhitungan dimulai dari 0, sedangkan apabila tidak ditemukan, program menampilkan tulisan **"TIDAK ADA"** sesuai ketentuan soal. Penggunaan algoritma Binary Search membuat proses pencarian jauh lebih efisien dibandingkan pencarian sekuensial karena setiap iterasi selalu membuang sekitar setengah dari ruang pencarian, sehingga kompleksitas waktunya menjadi **$O(\log n)$** , yang sangat sesuai untuk menangani jumlah data yang sangat besar hingga satu juta elemen sebagaimana ditentukan pada soal.